Lenguajes de Programación, Motor de Base de Datos y Métricas de Calidad del Sistema de Información y Control del Transmetro

Luis Orozco

Carné 3590 22 5339

Universidad Mariano Gálvez de Guatemala

Facultad de Ingeniería en Sistemas

Aseguramiento de la Calidad del Software

2 de octubre de 2026

Lenguajes de Programación, Motor de Base de Datos y Métricas de Calidad

El Sistema de Información y Control del Transmetro (SIC Transmetro) centraliza la información de 7 líneas, 106 estaciones y 210 unidades, hoy dispersa entre municipalidades y operadores. Este documento especifica los lenguajes de programación empleados, el motor de base de datos seleccionado con su justificación técnica y el plan de métricas de calidad del proyecto, junto con los valores medidos al 2 de octubre de 2026.

Lenguajes de Programación

El sistema emplea un solo lenguaje de programación, JavaScript, tanto en el servidor como en el navegador, además de SQL y PL/pgSQL en la base de datos. Esta decisión reduce el costo de cambio de contexto para el equipo de desarrollo: quien mantiene una capa puede leer la otra sin aprender un lenguaje adicional.

Tabla 1

Lenguajes empleados en el sistema y su justificación

Lenguaje	**Estándar o versión**	**Dónde se usa**	**Justificación**
JavaScript	ECMAScript 2022, módulos ES	API REST sobre Node.js 20 o superior	Mismo lenguaje que el frontend; Node.js ofrece un modelo de entrada y salida no bloqueante, adecuado para una API que espera respuestas de la base de datos
JavaScript con JSX	ECMAScript 2022 más JSX	Aplicación web en React 18	JSX permite construir la interfaz como componentes reutilizables
SQL	SQL:2016, dialecto PostgreSQL	Definición del esquema, consultas y vistas	Es el lenguaje nativo del motor; se escribe sin mapeador objeto-relacional para mantener visibles las restricciones y los disparadores
PL/pgSQL	PostgreSQL 14 o superior	15 funciones y 29 disparadores de reglas de negocio	Lenguaje procedimental nativo que ejecuta la regla dentro de la misma transacción del cambio
HTML5 y CSS3	Estándar vigente del W3C	Página contenedora y hoja de estilos	Hoja de estilos propia de 369 líneas, sin dependencias externas que mantener
JSON	RFC 8259	Intercambio de datos de la API y configuración	Formato estándar en servicios REST; PostgreSQL lo almacena de forma nativa como jsonb
Mermaid	Versión 11	22 diagramas del proyecto	Los diagramas se versionan y comparan como texto, no como imágenes

Nota. Elaboración propia a partir del código fuente del proyecto.

Alternativas Evaluadas para el Servidor

Se evaluaron cuatro alternativas antes de seleccionar JavaScript sobre Node.js. El criterio principal fue la cantidad de lenguajes y cadenas de herramientas que el equipo debe mantener en paralelo.

Tabla 2

Alternativas de lenguaje de servidor evaluadas

Alternativa	**Razón del descarte**
C# con ASP.NET Core	Plataforma sólida, pero obliga a mantener dos lenguajes y dos cadenas de herramientas, dado que el frontend permanecería en JavaScript
Java con Spring Boot	Requiere más configuración de la que justifica un sistema de este tamaño
Python con Django o FastAPI	Django impone su propio mapeador objeto-relacional, lo cual entra en conflicto con la decisión de ubicar las reglas de negocio en la base de datos
PHP con Laravel	Su ecosistema orienta la lógica hacia el modelo de la aplicación y no hacia el motor de datos

Nota. Elaboración propia.

Motor de Base de Datos

El sistema utiliza PostgreSQL en su versión 14 o superior, desplegado como servicio administrado en la nube. El script de creación se ejecutó y verificó por completo en PostgreSQL 18.3.

El requerimiento del proyecto exigía un motor relacional de código abierto. Entre los candidatos, PostgreSQL es el único que ofrece las tres características sobre las que descansa el diseño de este sistema: restricciones de exclusión sobre rangos temporales, columnas generadas y disparadores con un lenguaje procedimental completo (PostgreSQL Global Development Group, s.f.).

Tabla 3

Características del motor utilizadas por el sistema

Característica	**Uso en el sistema**	**Disponibilidad en MySQL o MariaDB**
EXCLUDE USING gist sobre rangos de tiempo	Impedir que una unidad esté asignada a dos líneas o a dos parqueos en periodos traslapados (RF-04 y RF-06)	No disponible; requeriría emulación mediante bloqueos en la aplicación
Columnas generadas y almacenadas	Calcular porcentaje de ocupación, sobrecupo y diferencia de conciliación (RF-09 y RF-12)	Disponible desde la versión 5.7, con menor cobertura de expresiones
Disparadores con lenguaje procedimental	Abrir y cerrar alertas automáticas y registrar la bitácora (RF-05, RF-08 y RF-16)	Limitados
Tipos ENUM, jsonb, inet y citext	Dominios cerrados, datos de auditoría, direcciones de red y correos sin distinción de mayúsculas	Parcial
Índices parciales y por expresión	Garantizar una sola asignación vigente por unidad y una sola alerta abierta por tipo y entidad	No disponible

Nota. RF corresponde a requerimiento funcional según el documento de especificación del proyecto.

Alternativas de Motor Descartadas

Tabla 4

Motores de base de datos descartados

Motor	**Razón del descarte**
SQL Server	No es de código abierto y su licencia de producción tiene costo
Oracle Database	Igual restricción de licencia y capacidades muy superiores a las requeridas
SQLite	Carece de concurrencia real de escritura y de gestión de usuarios
MongoDB u otro motor documental	El dominio es fuertemente relacional; sin integridad referencial, las reglas regresarían a la aplicación

Nota. Elaboración propia.

Configuración del Motor

Tabla 5

Decisiones de configuración de la base de datos

Aspecto	**Decisión adoptada**
Esquema	Todos los objetos residen en el esquema transmetro y no en public, para separarlos de las extensiones y permitir recrearlo por completo
Extensiones	btree_gist para las restricciones de exclusión, citext para usuario y correo, y pgcrypto para funciones de hash
Control de acceso	Un solo usuario de aplicación con permisos acotados; a la bitácora se le conceden únicamente INSERT y SELECT
Conexión	Agrupación de hasta 10 conexiones, cifrado TLS obligatorio y search_path fijado a transmetro, public
Transacciones	Toda escritura ocurre dentro de una transacción que publica el usuario mediante SET LOCAL app.usuario_id
Concurrencia	Bloqueo de fila con SELECT FOR UPDATE donde existe riesgo de condición de carrera, con las restricciones EXCLUDE como garantía final
Migraciones	Scripts numerados y versionados en la carpeta 01_database, ejecutables en orden

Nota. Elaboración propia a partir de los archivos de configuración del proyecto.

Métricas del Producto

Las mediciones se realizaron el 2 de octubre de 2026 sobre el repositorio del proyecto, excluyendo dependencias externas y archivos generados. El conteo de líneas incluye comentarios y líneas en blanco.

Tabla 6

Tamaño del código fuente por lenguaje

Lenguaje	**Archivos**	**Líneas**	**Participación**
JavaScript (servidor y utilidades)	40	3,532	35 %
JavaScript con JSX (interfaz)	18	2,278	23 %
SQL y PL/pgSQL	12	2,212	22 %
Mermaid (diagramas)	22	1,578	16 %
CSS	1	369	4 %
HTML y JSON	3	71	menos de 1 %
Total	96	10,040	100 %

Nota. Se excluyen las dependencias de terceros y los archivos generados por la compilación. La documentación en formato Markdown suma 1,103 líneas adicionales.

Tabla 7

Tamaño del modelo de datos

Elemento	**Cantidad**
Tablas	32
Vistas	12
Tipos enumerados	31
Funciones en PL/pgSQL	15
Disparadores	29
Índices creados explícitamente	37
Índices totales, incluidos los de llaves	104
Llaves foráneas	76
Restricciones CHECK	47
Restricciones UNIQUE	28
Restricciones EXCLUDE	4
Columnas generadas	4

Nota. Valores obtenidos mediante consulta al catálogo del sistema tras ejecutar el script completo.

Tabla 8

Tamaño de la interfaz de programación y de la aplicación web

Elemento	**Cantidad**
Puntos de acceso REST	51
Módulos del servidor	9
Pantallas de la aplicación web	11
Roles con permisos diferenciados	5
Módulos protegidos por la matriz de permisos	8

Nota. Elaboración propia a partir del código fuente.

Métricas de Calidad

El plan de métricas se organiza en métricas de producto y métricas de proceso, siguiendo la distinción entre características del producto de software y características de su construcción (Pressman y Maxim, 2020). Las características de calidad evaluadas se alinean con el modelo de calidad del producto de la norma ISO/IEC 25010 (ISO/IEC, 2011), en particular las de funcionalidad, eficiencia de desempeño, fiabilidad, seguridad y mantenibilidad.

Tabla 9

Métricas de producto del proyecto

Métrica	**Fórmula**	**Meta**	**Instrumento**	**Frecuencia**
Cobertura de pruebas en servicios	Líneas ejecutadas por las pruebas entre líneas totales de los módulos	Igual o mayor a 70 %	Vitest con reporte de cobertura	Al cierre de cada sprint
Cobertura de requerimientos	Requerimientos con al menos un caso de prueba automatizado entre requerimientos totales	100 % de los requerimientos con regla de negocio	Matriz de trazabilidad	Al cierre de cada sprint
Densidad de defectos	Defectos encontrados entre miles de líneas de código	Menos de 5 por cada mil líneas	Registro de defectos	Al cierre de cada sprint
Defectos escapados	Defectos hallados después de la entrega entre defectos totales	Menos del 10 %	Registro de defectos	Al final del proyecto
Reglas garantizadas en la base de datos	Reglas con restricción o disparador entre reglas de negocio totales	100 % de las reglas duras	Revisión del script	Al cierre de cada sprint
Tiempo de respuesta	Percentil 95 de los diez puntos de acceso más utilizados	Menos de 3 segundos	Prueba de carga con k6	Sprint 7 y previo a la entrega
Disponibilidad	Minutos en servicio entre minutos del horario operativo	Igual o mayor a 99 %	Sonda automática al punto de acceso de salud	Mensual

Nota. Las metas de tiempo de respuesta y disponibilidad provienen de los requerimientos no funcionales del proyecto.

Tabla 10

Métricas de proceso del proyecto

Métrica	**Fórmula**	**Propósito**
Velocidad del equipo	Historias de usuario terminadas por sprint	Proyectar si el alcance restante cabe en el tiempo disponible
Cumplimiento de la definición de terminado	Historias que cumplen los siete criterios entre historias cerradas	Detectar avance aparente que en realidad acumula deuda técnica
Defectos por fase de detección	Defectos hallados en diseño, en pruebas y en producción	Sustentar que el costo de corrección crece conforme el defecto avanza de fase
Retrabajo	Horas dedicadas a corrección entre horas totales	Cuantificar el costo de la mala calidad
Tiempo medio de corrección	Promedio entre el reporte de un defecto y su cierre	Evaluar la capacidad de respuesta del equipo

Nota. Elaboración propia con base en Pressman y Maxim (2020).

Estado Actual de las Métricas

Los valores siguientes corresponden al cierre del Sprint 0.

Tabla 11

Valores medidos al 2 de octubre de 2026

Métrica	**Valor actual**	**Meta**	**Estado**
Pruebas automatizadas	9, todas satisfactorias	Incremento por sprint	En curso
Cobertura de pruebas en servicios	Sin medir; las pruebas actuales cubren autenticación y permisos	Igual o mayor a 70 %	Pendiente
Reglas de negocio garantizadas en la base	7 de 7 reglas duras	100 %	Cumplido
Reglas verificadas con datos inválidos	9 pruebas ejecutadas contra PostgreSQL 18.3, todas con el resultado esperado	100 % de las reglas duras	Cumplido
Requerimientos implementados	7 de 16	16 de 16	En curso
Trazabilidad de requerimientos	16 de 16 documentados en la matriz	100 %	Cumplido
Defectos encontrados y corregidos	2	Registro completo	En curso
Tiempo de respuesta	Sin medir	Menos de 3 segundos	Pendiente

Nota. Las reglas duras corresponden a los requerimientos funcionales RF-04, RF-05, RF-06, RF-08, RF-09, RF-10 y RF-16.

Procedimiento de Recolección

Las métricas automatizables se obtienen mediante los comandos siguientes, ejecutados desde la carpeta del servidor:

```
npm test                    Ejecuta las pruebas automatizadas
npx vitest run --coverage   Genera el reporte de cobertura
npm run db:reset            Recrea el esquema y verifica el script
completo
```

La verificación del script contra un motor real se realiza con PGlite, una compilación de PostgreSQL que se ejecuta sin instalar un servidor. Esto permite correr las pruebas de reglas de negocio en cualquier equipo y, posteriormente, dentro de un proceso de integración continua.

Conclusiones

El sistema emplea JavaScript como único lenguaje de programación de aplicación, con SQL y PL/pgSQL en la base de datos, lo cual reduce la cantidad de tecnologías que el equipo debe dominar. El motor seleccionado es PostgreSQL 14 o superior, cuya elección responde a una necesidad técnica verificable: es el único motor relacional de código abierto que ofrece restricciones de exclusión sobre rangos temporales, mecanismo que garantiza los requerimientos RF-04 y RF-06 sin depender del código de la aplicación.

Al cierre del Sprint 0, el producto suma 10,040 líneas de código, 32 tablas, 51 puntos de acceso y 11 pantallas. Siete de las siete reglas de negocio duras están garantizadas por la base de datos y verificadas con datos inválidos. Quedan pendientes de medición la cobertura de pruebas y el tiempo de respuesta, ambas programadas conforme avancen los sprints.

Referencias

Congreso de la República de Guatemala. (2008). *Decreto 57-2008, Ley de Acceso a la Información Pública*. Diario de Centro América.

ISO/IEC. (2011). *ISO/IEC 25010:2011. Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — System and software quality models*. International Organization for Standardization.

OpenJS Foundation. (s.f.). *Node.js documentation*. https://nodejs.org/docs

PostgreSQL Global Development Group. (s.f.). *PostgreSQL documentation*. https://www.postgresql.org/docs/

Pressman, R. S., y Maxim, B. R. (2020). *Software engineering: A practitioner's approach* (9.ª ed.). McGraw-Hill Education.

Sommerville, I. (2016). *Software engineering* (10.ª ed.). Pearson Education.